

SINERGYFIT

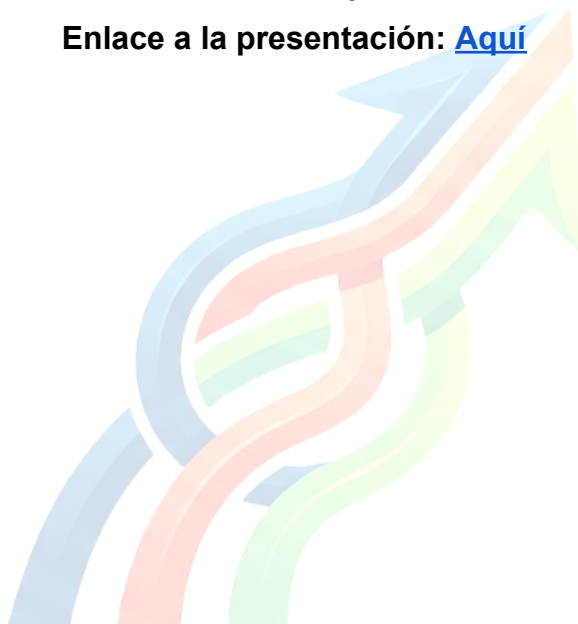
Sistema de Seguimiento y Gestión de Actividad Física

Autor: Juan Francisco Cortejosa Galindo

Fecha: Mayo de 2026

Tutor: Marta Lopez Ron

Enlace a la presentación: [Aquí](#)



ÍNDICE DE CONTENIDOS

ÍNDICE DE CONTENIDOS	2
HOJA RESUMEN	4
1.- INTRODUCCIÓN	5
2.- ESTUDIO DE LA VIABILIDAD	7
2.1. Diagnóstico de la situación actual	7
2.2. Viabilidad del proyecto	7
3.- ALTERNATIVAS Y SELECCIÓN DE LA SOLUCIÓN	9
3.1. Alternativas de Arquitectura Backend	9
3.2. Alternativas de Desarrollo Frontend	9
3.3. Alternativas de Entorno y Despliegue	10
4.- ANÁLISIS Y DISEÑO DE LA SOLUCIÓN ADOPTADA	11
4.1. ANÁLISIS DE REQUISITOS DEL SISTEMA	11
4.2. DISEÑO DE LA BASE DE DATOS (MODELO E/R)	12
4.3. DISEÑO DE LA INTERFAZ DE USUARIO (UI/UX)	12
4.4. ARQUITECTURA DEL SISTEMA	13
5.- IMPLEMENTACIÓN Y DESARROLLO	14
5.1. FASE 1: MAQUETACIÓN Y FRONTEND INTERACTIVO (SPRINTS 1 Y 2)	14
5.2. FASE 2: BACKEND NATIVO Y PERSISTENCIA DE DATOS (SPRINT 3)	15
5.3. FASE 3: MIGRACIÓN A LARAVEL Y ARQUITECTURA MVC (SPRINTS 4, 5 Y 6)	15
5.4. FASE 4: LÓGICA AVANZADA, GAMIFICACIÓN Y UX (SPRINTS 7 Y 8)	16
5.5. FASE 5: CONTENERIZACIÓN Y DESPLIEGUE (SPRINT 9)	17
6.- PRUEBAS Y CONTROL DE CALIDAD	19
6.1. PRUEBAS DE LÓGICA DE NEGOCIO (BACKEND)	19
6.2. PRUEBAS DE INTERFAZ Y USABILIDAD (FRONTEND)	19
7.- COSTES Y PRESUPUESTO	21
7.1. COSTES DE RECURSOS HUMANOS	21
7.2. COSTES DE SOFTWARE (LICENCIAS)	21
7.3. COSTES DE INFRAESTRUCTURA (DESPLIEGUE ANUAL)	22
7.4. PRESUPUESTO TOTAL ESTIMADO	22
8.- CONCLUSIONES	23
8.1. GRADO DE CUMPLIMIENTO DE LOS OBJETIVOS	23
8.2. DIFICULTADES ENCONTRADAS	23
8.3. AMPLIACIONES FUTURAS	23
8.4. VALORACIÓN PERSONAL	24
9.- BIBLIOGRAFÍA Y WEBGRAFÍA	25
10.- GLOSARIO	26
11.- ANEXOS	27

HOJA RESUMEN

Título del proyecto: SinergyFit Autor: Juan Francisco Cortejosa Galindo

Tutor: Marta Lopez Ron

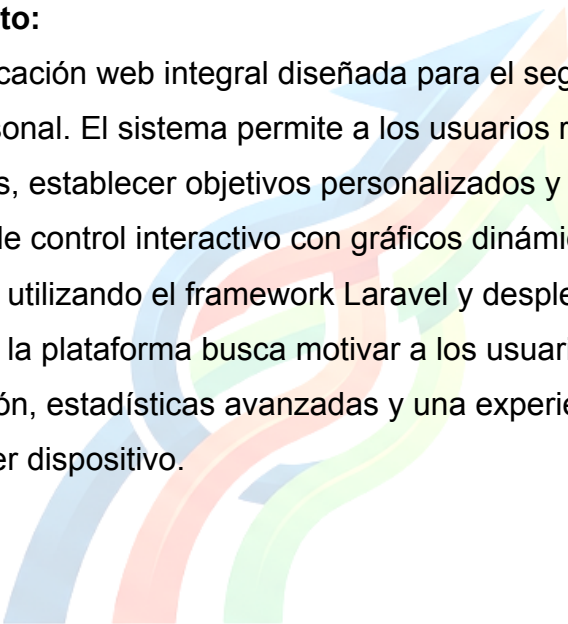
Fecha: Mayo de 2026

Ciclo Formativo: Desarrollo de Aplicaciones Web (DAW) Curso: 2º Curso

Palabras clave: Fitness, Salud, Laravel, Aplicación Web, Docker, Tracking, Rendimiento, Desarrollo Ágil, PHP, MySQL.

Resumen del proyecto:

SinergyFit es una aplicación web integral diseñada para el seguimiento y gestión de la actividad física personal. El sistema permite a los usuarios registrar sus entrenamientos diarios, establecer objetivos personalizados y visualizar su progreso a través de un panel de control interactivo con gráficos dinámicos. Desarrollada bajo una arquitectura MVC utilizando el framework Laravel y desplegada en contenedores Docker, la plataforma busca motivar a los usuarios mediante un sistema de gamificación, estadísticas avanzadas y una experiencia de usuario fluida y adaptable a cualquier dispositivo.



1.- INTRODUCCIÓN

En la sociedad actual, el sedentarismo y la falta de actividad física constante se han convertido en un problema de salud pública de primer orden. Al mismo tiempo, la tecnología y el uso de dispositivos digitales forman parte intrínseca de nuestro día a día. Partiendo de esta premisa, nace el proyecto SinergyFit (inicialmente concebido bajo el nombre de FitnessTracker), una aplicación web orientada a fusionar la tecnología con el bienestar y el proyecto personal.

El propósito central de SinergyFit es ofrecer una herramienta accesible, intuitiva y robusta que ayude a las personas a mantener un registro exhaustivo de su actividad física y a mantenerse motivadas a largo plazo.

Para lograr este propósito, el proyecto se ha diseñado con los siguientes objetivos generales:

- **Gestión Integral de Entrenamientos:** Permitir a los usuarios registrar de forma detallada sus sesiones de ejercicio, incluyendo métricas clave como el tipo de actividad, duración, intensidad y volumen de trabajo.
- **Visualización del Progreso:** Ofrecer un panel de control (Dashboard) dinámico que convierta los datos crudos en gráficos estadísticos visuales y fáciles de interpretar.
- **Motivación y Retención:** Implementar un sistema de objetivos personales y mecánicas de gamificación (logros y rachas) que incentiven al usuario a mantener una rutina constante.

A nivel personal y académico, la motivación principal que me ha llevado a decantarme por este proyecto ha sido la oportunidad de aplicar y consolidar los conocimientos adquiridos durante el Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Web. Este proyecto no solo abarca el diseño e implementación de interfaces interactivas, sino que ha supuesto un desafío arquitectónico, escalando desde un desarrollo inicial estático hasta una aplicación robusta soportada por el framework Laravel (PHP), bases de datos relacionales (MySQL) y tecnologías modernas de despliegue y virtualización (Docker).

En definitiva, SinergyFit no es únicamente una plataforma orientada a la salud de sus usuarios, sino que representa el resultado final de un ciclo de aprendizaje iterativo, desarrollado a través de múltiples Sprints bajo metodologías ágiles,

abarcando todas las fases del ciclo de vida del software, desde el análisis de requisitos inicial hasta su puesta en producción.



2.- ESTUDIO DE LA VIABILIDAD

2.1. Diagnóstico de la situación actual

El bienestar físico y la salud son preocupaciones crecientes en la sociedad moderna. A pesar de la amplia disponibilidad de gimnasios y centros deportivos, muchas personas encuentran dificultades para mantener la constancia en sus entrenamientos debido a la falta de una planificación clara y a la incapacidad de visualizar su progreso a lo largo del tiempo.

Habitualmente, los usuarios que intentan registrar sus rutinas lo hacen mediante métodos ineficientes o poco escalables, como libretas de papel, notas en el teléfono móvil o complejas hojas de cálculo. Estos métodos carecen de interactividad, no ofrecen cálculos automáticos de métricas (como calorías quemadas o volumen total de peso levantado) y, lo más importante, no proporcionan retroalimentación visual ni motivación.

Bajo este diagnóstico surge la necesidad de desarrollar SinergyFit: una solución centralizada, digital e interactiva que permita a los usuarios no solo registrar sus datos, sino transformarlos en información visual útil para la consecución de sus objetivos de salud.

2.2. Viabilidad del proyecto

Para asegurar que el desarrollo de SinergyFit pudiera llevarse a cabo con éxito dentro del marco del Proyecto Intermodular, se realizó un estudio de viabilidad abarcando tres dimensiones principales:

- **Viabilidad Técnica:** El proyecto se ha planteado utilizando un stack tecnológico sólido y ampliamente documentado. El uso de lenguajes y herramientas como HTML5, CSS3, JavaScript, PHP y MySQL garantiza que el desarrollo sea viable. Además, la posterior adopción del framework Laravel permite estructurar el código bajo el patrón MVC, asegurando mantenibilidad y escalabilidad. Las competencias técnicas requeridas se alinean perfectamente con los Resultados de Aprendizaje (RA) del ciclo formativo.
- **Viabilidad Económica:** Al basarse íntegramente en tecnologías de código abierto (Open Source), los costes de licencias de software son nulos. Las

herramientas de desarrollo (VS Code, Docker, Git) son gratuitas, lo que hace que el proyecto sea 100% viable y económica desde su fase de desarrollo hasta un posible despliegue en producción inicial.

- Viabilidad Operativa: El diseño responsivo de la aplicación asegura que pueda ser utilizada tanto desde ordenadores de escritorio como desde dispositivos móviles, adaptándose al entorno real en el que un usuario registraría sus entrenamientos (por ejemplo, desde el propio gimnasio).



3.- ALTERNATIVAS Y SELECCIÓN DE LA SOLUCIÓN

Durante las fases iniciales de diseño , se evaluaron diferentes enfoques tecnológicos y arquitectónicos para abordar el desarrollo de SinergyFit. Tras un análisis detallado, se seleccionaron las opciones que mejor garantizaban el rendimiento, la seguridad y el cumplimiento de los plazos.

3.1. Alternativas de Arquitectura Backend

- Alternativa A (PHP Puro): Desarrollar toda la lógica de servidor utilizando scripts de PHP nativo y consultas SQL directas mediante PDO. Aunque es un enfoque válido para proyectos pequeños y se utilizó durante el Sprint 3 para asentar conocimientos, carece de estructura a largo plazo, dificultando el mantenimiento y la seguridad.
- Alternativa B (Framework Laravel): Utilizar un framework moderno de PHP. Laravel proporciona una estructura robusta basada en el patrón Modelo-Vista-Controlador (MVC), un ORM potente (Eloquent) para interactuar con la base de datos sin escribir SQL directo, y sistemas integrados de enrutamiento y autenticación.
- Decisión Adoptada: Se optó por la Alternativa B (Laravel) a partir del Sprint 4. Esta transición permitió agilizar el desarrollo, mejorar drásticamente la seguridad del sistema y organizar el código de manera profesional, preparando la aplicación para futuras integraciones.

3.2. Alternativas de Desarrollo Frontend

- Alternativa A (Framework JS React/Vue): Separar completamente el Frontend del Backend creando una API REST en Laravel y consumiéndola desde un framework de JavaScript (Single Page Application). Aunque es un enfoque muy potente, aumentaba la complejidad del proyecto y el tiempo de desarrollo de manera desproporcionada para los objetivos del ciclo.
- Alternativa B (Plantillas Blade + Bootstrap + Vanilla JS): Renderizar las vistas directamente desde el servidor utilizando el motor de plantillas Blade de

Laravel, apoyándose en el framework CSS Bootstrap 5 para el diseño responsivo y JavaScript puro (junto a librerías como Chart.js) para la interactividad específica de los gráficos y validaciones.

- Decisión Adoptada: Se optó por la Alternativa B. Esta solución permitió una integración perfecta y rápida con el backend, manteniendo una excelente experiencia de usuario y cumpliendo con los requisitos de diseño responsivo estipulados desde el Sprint 2.

3.3. Alternativas de Entorno y Despliegue

- Alternativa A (Entornos locales tradicionales): Utilizar paquetes como XAMPP o WAMP. Si bien son útiles para las primeras pruebas, a menudo generan problemas de incompatibilidad de versiones al pasar el código de un ordenador a otro (el clásico problema de "en mi máquina funciona").
- Alternativa B (Contenedores con Docker): Utilizar Docker y Docker Compose para virtualizar el entorno de desarrollo y producción. Esto garantiza que la aplicación, la base de datos y el servidor web se ejecuten exactamente bajo las mismas condiciones independientemente de la máquina host.
- Decisión Adoptada: Se seleccionó la Alternativa B (Docker). Su implementación en el Sprint 9 aseguró un entorno estandarizado, profesional y listo para despliegues en producción de manera controlada.

4.- ANÁLISIS Y DISEÑO DE LA SOLUCIÓN ADOPTADA

Antes de comenzar la fase de desarrollo y codificación, se llevó a cabo una etapa exhaustiva de análisis y diseño. El objetivo de esta fase fue definir claramente qué iba a hacer el sistema, cómo iba a almacenar la información y cómo interactuaría el usuario con él.

4.1. ANÁLISIS DE REQUISITOS DEL SISTEMA

A partir del estudio inicial, se definieron los requisitos funcionales y no funcionales de SinergyFit. Estos se organizaron en un Product Backlog que guió todo el desarrollo ágil.

Requisitos Funcionales Principales:

- **Gestión de Usuarios:** El sistema debe permitir el registro, inicio de sesión y cierre de sesión de usuarios de forma segura. Cada usuario debe tener un perfil asociado con sus datos físicos (peso, altura, nivel de actividad).
- **Registro de Entrenamientos:** Los usuarios deben poder registrar nuevas sesiones de entrenamiento, especificando el tipo de ejercicio, la duración, las repeticiones, las series y el peso levantado.
- **Visualización y Seguimiento:** El sistema debe proveer un Dashboard (panel de control) personal donde el usuario pueda ver un resumen de su actividad reciente y gráficos de progreso temporal.
- **Sistema de Objetivos:** Debe existir la posibilidad de establecer metas personales (por ejemplo, entrenar 3 días a la semana) y que el sistema calcule automáticamente el porcentaje de progreso.
- **Gestión del Calendario:** Los entrenamientos deben reflejarse en un calendario interactivo para facilitar la visualización del historial a nivel mensual.

Requisitos No Funcionales:

- **Diseño Responsivo:** La interfaz debe adaptarse a dispositivos móviles, tablets y ordenadores de escritorio.

- Seguridad: Las contraseñas deben almacenarse encriptadas y el acceso a los paneles privados debe estar protegido mediante middleware de autenticación.
- Rendimiento: Las consultas a la base de datos deben ser eficientes, apoyándose en la carga asíncrona (AJAX) para mejorar la experiencia de usuario.

4.2. DISEÑO DE LA BASE DE DATOS (MODELO E/R)

Para estructurar la persistencia de datos, se diseñó un modelo relacional normalizado que asegura la integridad de la información y evita la redundancia. Las entidades principales que componen el sistema son:

1. Usuarios: Almacena las credenciales de acceso (email, contraseña encriptada) y los datos de perfil físico necesarios para los cálculos de métricas (peso, altura).
2. Entrenamientos: Actúa como la cabecera de una sesión de ejercicio. Contiene la fecha, duración total y está vinculada a un usuario específico.
3. Ejercicios: Representa el detalle de un entrenamiento. Un entrenamiento puede tener múltiples ejercicios asociados. Guarda información sobre el nombre del ejercicio, número de series, repeticiones y peso.
4. Objetivos/Metas: Almacena las metas fijadas por el usuario, estableciendo fechas límite y valores objetivo.

(Relaciones: Un usuario puede tener muchos entrenamientos y objetivos (1:N). Un entrenamiento puede contener varios ejercicios registrados (1:N)).

4.3. DISEÑO DE LA INTERFAZ DE USUARIO (UI/UX)

El diseño de la interfaz se planteó con el objetivo de ser motivador y fácil de interpretar. Durante la fase de maquetación se tomaron las siguientes decisiones de diseño:

- Wireframes: Se trazaron bocetos iniciales para definir la disposición de los elementos. Se determinó que la vista principal (Dashboard) debía contener "tarjetas" (cards) resumen en la parte superior y gráficos de tendencias en el centro de la pantalla.

- Paleta de colores y Tipografía: Se eligieron colores enérgicos y modernos propios de aplicaciones deportivas (tonos oscuros combinados con colores de acento vivos) y tipografías sin serifa para garantizar la legibilidad en pantallas pequeñas.
- Librerías Gráficas: Se integró la librería Chart.js para renderizar gráficos visualmente atractivos (de barras y de líneas) que muestran la evolución calórica y el volumen de entrenamiento.

4.4. ARQUITECTURA DEL SISTEMA

La arquitectura adoptada para la solución final se basa en el patrón Modelo-Vista-Controlador (MVC), sustentado por el framework Laravel y encapsulado en contenedores Docker:

- Modelo (Model): Representado por las clases de Eloquent ORM. Se encargan de la lógica de negocio y de interactuar con la base de datos MySQL de forma orientada a objetos.
- Vista (View): Formulada con plantillas Blade. Combinan código HTML, CSS (mediante Bootstrap) y directivas de Laravel para mostrar los datos al usuario de manera dinámica.
- Controlador (Controller): Clases PHP que actúan como intermediarios. Reciben las peticiones HTTP del usuario, solicitan los datos necesarios a los Modelos, aplican validaciones de seguridad y envían la respuesta a las Vistas correspondientes.
- Infraestructura: Todo este ecosistema se levanta sobre un entorno Docker, garantizando que el servidor web (Nginx/Apache), el procesador PHP y el motor de base de datos MySQL funcionen en entornos aislados pero comunicados a través de una red virtual interna.

5.- IMPLEMENTACIÓN Y DESARROLLO

El desarrollo de SinergyFit no se llevó a cabo de forma monolítica, sino que siguió una metodología incremental dividida en Sprints. Esto permitió construir el sistema capa por capa, comenzando por la interfaz visual estática hasta llegar a un backend complejo y contenedorizado.

A continuación, se detalla el proceso de implementación técnica dividido en las fases principales del proyecto.

5.1. FASE 1: MAQUETACIÓN Y FRONTEND INTERACTIVO

El primer paso lógico en el desarrollo fue la construcción de la capa de presentación. Antes de escribir cualquier código de servidor, se necesitaba una interfaz funcional que pudiera simular la experiencia final del usuario.

- Estructura HTML5 y CSS3: Se construyeron las vistas principales utilizando etiquetas semánticas de HTML5 (<header>, <nav>, <main>, <section>). Para agilizar el proceso de diseño y garantizar que la aplicación fuera 100% responsive (adaptable a móviles), se integró el framework CSS Bootstrap 5. Esto permitió crear sistemas de rejillas (grids), tarjetas de contenido (cards) y modales interactivos con muy pocas líneas de CSS personalizado.
- Interactividad con JavaScript Vanilla: Para dar vida a los formularios antes de tener una base de datos real, se utilizó JavaScript puro. Se programaron validaciones de lado del cliente para evitar que el usuario introdujera datos erróneos (por ejemplo, pesos negativos o correos mal formateados). Temporalmente, los datos introducidos se guardaban en el LocalStorage del navegador para simular el registro de los entrenamientos.
- Integración de Gráficos (Chart.js): Una de las piezas clave del Dashboard era la visualización de datos. Se importó la librería Chart.js mediante CDN. Se desarrollaron scripts en JavaScript que tomaban los datos (inicialmente simulados) y los renderizaban en formato de gráficos de barras para las calorías quemadas y gráficos de líneas para la progresión del volumen de peso levantado a lo largo de la semana.

5.2. FASE 2: BACKEND NATIVO Y PERSISTENCIA DE DATOS (SPRINT 3)

Una vez que el Frontend era visualmente funcional, el siguiente objetivo fue dotar a la aplicación de persistencia real mediante una base de datos y un servidor capaz de procesar la información.

- Implementación de la Base de Datos: Se procedió a materializar el modelo Entidad/Relación previamente diseñado creando las tablas en MySQL mediante la herramienta phpMyAdmin. Se definieron las claves primarias, claves foráneas y los tipos de datos adecuados (por ejemplo, VARCHAR para correos, INT para repeticiones y DECIMAL para pesos).
- Desarrollo en PHP Puro: En esta etapa formativa, se programó un sistema CRUD (Crear, Leer, Actualizar, Borrar) utilizando PHP nativo. Se estableció la conexión a la base de datos utilizando la extensión PDO (PHP Data Objects), lo cual proporcionó una capa extra de seguridad al permitir el uso de sentencias preparadas (Prepared Statements) para proteger el sistema contra ataques de inyección SQL.

A pesar de lograr un funcionamiento correcto, se evidenció que mantener archivos PHP puros mezclados con HTML generaba un "código espagueti" difícil de escalar. Esto justificó el salto hacia un framework profesional en la siguiente fase.

5.3. FASE 3: MIGRACIÓN A LARAVEL Y ARQUITECTURA MVC (SPRINTS 4, 5 Y 6)

La evolución más significativa del proyecto fue la reescritura del backend utilizando Laravel. Esta migración transformó SinergyFit en una aplicación web moderna y estructurada.

- a) Configuración y Enrutamiento: Se inicializó el proyecto Laravel utilizando Composer. Se configuró el archivo routes/web.php para definir un sistema de rutas claro y semántico. Se sustituyó la navegación directa entre archivos por llamadas a controladores específicos (EntrenamientoController, DashboardController), separando completamente la lógica de la vista.
- b) Modelos y ORM Eloquent: Se sustituyeron las consultas SQL directas (PDO) por el sistema ORM de Laravel, Eloquent. Se crearon Modelos para cada tabla de la

base de datos (Usuario, Entrenamientos, Ejercicios). Además, se programaron las migraciones de Laravel, lo que permitió que la estructura de la base de datos se generara automáticamente desde el código PHP sin depender de phpMyAdmin, facilitando enormemente el control de versiones.

c) Vistas con Blade: Todo el código HTML desarrollado en la Fase 1 se migró al motor de plantillas Blade. Esto supuso una mejora brutal en la reutilización de código gracias a la herencia de plantillas (@extends, @section). Elementos repetitivos como la barra de navegación o el pie de página se separaron en componentes, limpiando enormemente el código de las vistas principales.

d) Autenticación y Seguridad: Para gestionar el acceso de los usuarios, se implementó Laravel Breeze. Este paquete proporcionó automáticamente un sistema robusto de inicio de sesión, registro y recuperación de contraseñas. Se configuraron Middlewares personalizados para proteger las rutas privadas, asegurando que un usuario anónimo fuera redirigido a la pantalla de login si intentaba acceder al Dashboard de SinergyFit.

5.4. FASE 4: LÓGICA AVANZADA, GAMIFICACIÓN Y UX (SPRINTS 7 Y 8)

Con la base de datos y la arquitectura MVC plenamente operativas, el esfuerzo de desarrollo se centró en añadir valor al usuario final mediante funcionalidades avanzadas y una mejora sustancial en la Experiencia de Usuario (UX).

a) Sistema de Calendario Interactivo: Para cumplir con el requisito de visualización histórica, se integró la librería Javascript FullCalendar.js. Mediante controladores específicos en Laravel, se enviaban los datos de los entrenamientos almacenados en la base de datos formateados en JSON. El calendario renderizaba estos datos en tiempo real, permitiendo al usuario ver qué días había entrenado de un solo vistazo.

b) Lógica de Gamificación y Objetivos: Se programó en el servidor la lógica matemática necesaria para analizar el progreso del usuario. Se implementó un sistema de cálculo que comparaba los entrenamientos realizados en una semana con las metas establecidas (ej. "Entrenar 4 días a la semana" o "Levantar 10,000 kg totales"). Dependiendo de estos resultados, la interfaz mostraba barras de progreso que cambiaban de color según el porcentaje de consecución, aplicando principios de gamificación para mantener la motivación.

c) Optimización Asíncrona (AJAX): Para evitar recargas innecesarias de la página que perjudicaran la fluidez (especialmente al borrar un entrenamiento o marcar un objetivo como completado), se implementaron peticiones asíncronas utilizando la API Fetch de JavaScript interactuando con rutas API de Laravel. Esto otorgó a SinergyFit una sensación de Single Page Application (SPA) en las vistas más interactivas.

5.5. FASE 5: CONTENERIZACIÓN Y DESPLIEGUE (SPRINT 9)

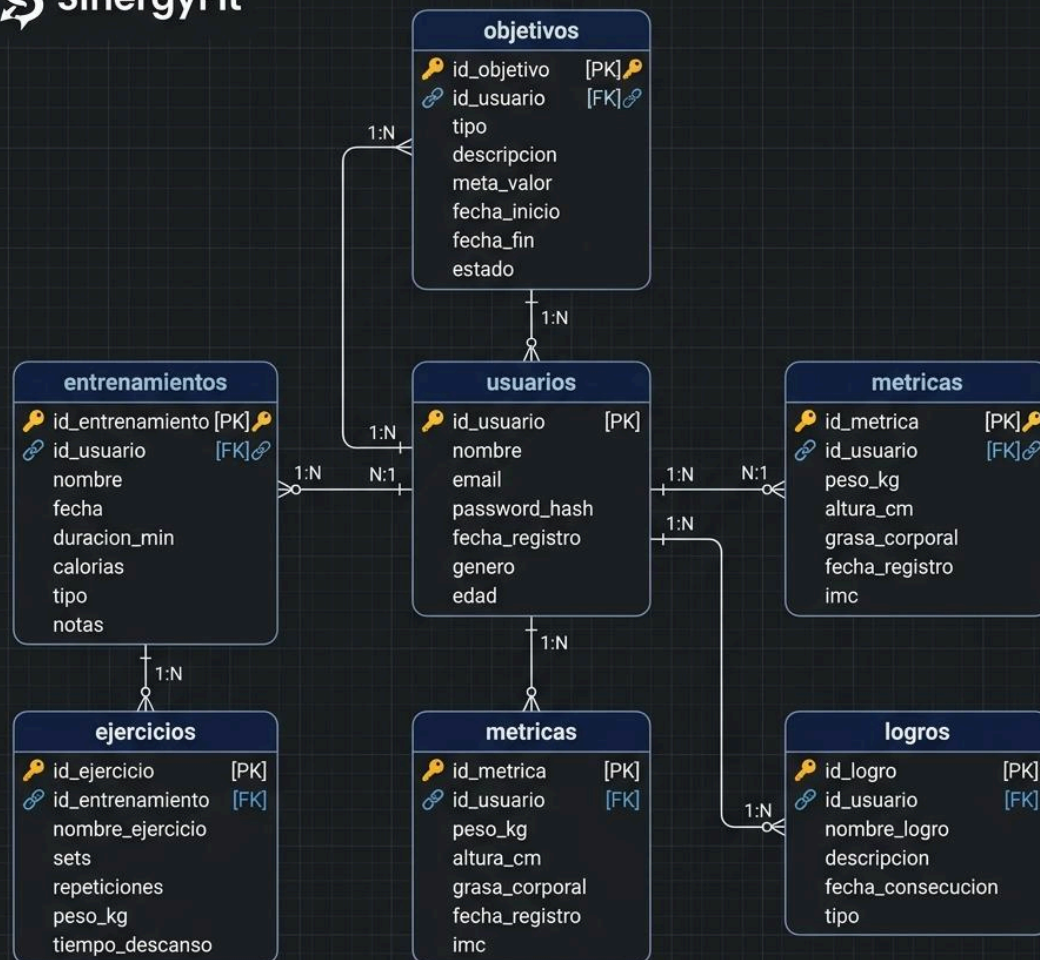
La última fase del proyecto consistió en preparar la aplicación para su distribución y despliegue en entornos de producción, abandonando las herramientas de desarrollo local tradicionales (como XAMPP) en favor de tecnologías de virtualización modernas.

a) Implementación de Docker: Para garantizar que SinergyFit se ejecutara de manera idéntica independientemente del sistema operativo host, se contenerizó la aplicación utilizando Docker. Se redactó un archivo Dockerfile personalizado que definía una imagen ligera de PHP con las extensiones necesarias instaladas (como pdo_mysql) y Composer integrado.

b) Orquestación con Docker Compose: Para manejar los múltiples servicios que requiere la aplicación, se configuró un archivo docker-compose.yml. Este archivo orquesta los siguientes contenedores interconectados:

1. Contenedor Web/App: Ejecutando PHP-FPM y el código fuente de Laravel.
2. Contenedor Servidor Web: Actuando como proxy inverso (Nginx o Apache) para servir los archivos estáticos y redirigir las peticiones PHP.
3. Contenedor de Base de Datos: Ejecutando la imagen oficial de MySQL 8.0, montando volúmenes persistentes para asegurar que los datos de los usuarios no se pierdan al reiniciar los contenedores.

SinergyFit | Database Schema



Esta fase de contenerización marcó el culmen técnico del proyecto, asegurando que cualquier desarrollador o servidor de producción pudiera desplegar SinergyFit en cuestión de minutos ejecutando un único comando, demostrando la madurez del ciclo de desarrollo seguido.

6.- PRUEBAS Y CONTROL DE CALIDAD

Para garantizar la estabilidad y fiabilidad de SinergyFit antes de su despliegue final (enfocado principalmente durante el Sprint 8), se diseñó e implementó una estrategia de pruebas (Testing) que abarcó tanto la lógica interna del servidor como la interfaz visual del usuario.

6.1. PRUEBAS DE LÓGICA DE NEGOCIO (BACKEND)

Dado que SinergyFit maneja cálculos matemáticos para las estadísticas de los usuarios (calorías quemadas, suma de pesos, porcentajes de progreso), era crítico asegurar que estas funciones no tuvieran errores.

- **Pruebas Unitarias:** Se utilizó el framework PHPUnit (integrado por defecto en Laravel) para probar aisladamente los métodos de los modelos. Se crearon tests para verificar que, al introducir ciertos parámetros de entrenamiento, el sistema devolviera el cálculo exacto esperado.
- **Depuración Avanzada:** Se integró la herramienta Laravel Debugbar en el entorno de desarrollo. Esto permitió monitorizar en tiempo real el consumo de memoria y, sobre todo, el número de consultas SQL ejecutadas por cada página. Gracias a esto, se detectaron y corrigieron problemas de "N+1 queries" (consultas redundantes a la base de datos), optimizando el rendimiento mediante el uso de Eager Loading en Eloquent.

6.2. PRUEBAS DE INTERFAZ Y USABILIDAD (FRONTEND)

Pruebas de Caja Negra: Se simularon escenarios de uso reales (crear un usuario nuevo, intentar registrar un entrenamiento con datos vacíos, intentar acceder a rutas protegidas sin estar logueado).

- **Validaciones:** Se comprobó exhaustivamente que tanto las validaciones del lado del cliente (HTML5/JS) como las del lado del servidor (Form Requests de Laravel) bloquearan correctamente cualquier intento de inyección de datos maliciosos.
- **Pruebas de Responsividad:** Se utilizó la herramienta de desarrollador del navegador para simular resoluciones de pantalla de distintos dispositivos

móviles, asegurando que los gráficos de Chart.js y las tablas de Bootstrap se redimensionaran correctamente sin romper el diseño.

[Opcional: Si tienes alguna captura de pantalla ejecutando PHPUnit en la terminal, o mostrando la barra roja de Laravel Debugbar, este es un sitio genial para ponerla]



7.- COSTES Y PRESUPUESTO

A la hora de plantear el desarrollo de un proyecto de software, es imprescindible realizar una evaluación económica para determinar el coste de producción y mantenimiento del producto final. El presupuesto de SinergyFit se ha dividido en tres categorías: Recursos Humanos, Costes de Software e Infraestructura.

7.1. COSTES DE RECURSOS HUMANOS

Para el cálculo del coste de desarrollo, se han tomado como referencia las estimaciones en horas del Product Backlog inicial (24 tareas principales). El total aproximado de desarrollo activo, sumando fases de análisis, maquetación, backend y despliegue, asciende a unas 160 horas de trabajo.

Asumiendo el rol de un desarrollador web Junior con una tarifa estándar estimada de 20€/hora, el desglose sería el siguiente:

- Fase de Análisis y Diseño (Sprints 1-2): 36 horas = 720 €
- Desarrollo Backend y Base de datos (Sprints 3-6): 64 horas = 1.280 €
- Lógica avanzada, Frontend y Testing (Sprints 7-8): 44 horas = 880 €
- Despliegue y Documentación (Sprint 9): 16 horas = 320 €
- Total Estimado de Mano de Obra: 3.200 €

7.2. COSTES DE SOFTWARE (LICENCIAS)

Una de las mayores ventajas de la arquitectura elegida para SinergyFit es su dependencia exclusiva de tecnologías de código abierto (Open Source).

- Sistema Operativo de desarrollo: Linux/Windows (0 €).
- IDE y Herramientas: Visual Studio Code, Git, Docker (0 €).
- Lenguajes y Frameworks: PHP 8, Laravel 10, HTML/CSS/JS, Bootstrap (0 €).
- Base de datos: MySQL (0 €).
- Total de Licencias de Software: 0 €

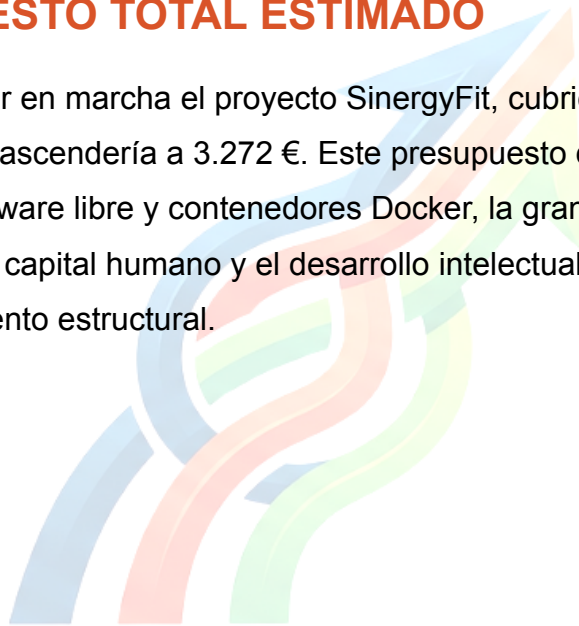
7.3. COSTES DE INFRAESTRUCTURA (DESPLIEGUE ANUAL)

Para alojar la aplicación contenerizada con Docker en producción, se requiere un servidor VPS (Servidor Privado Virtual) y un nombre de dominio.

- Servidor VPS (Linux, 2GB RAM, 40GB SSD): ~60 € / año.
- Registro de Dominio (.es / .com): ~12 € / año.
- Total de Infraestructura anual: 72 € / año

7.4. PRESUPUESTO TOTAL ESTIMADO

El coste total de poner en marcha el proyecto SinergyFit, cubriendo el primer año de vida de la aplicación, ascendería a 3.272 €. Este presupuesto demuestra que, gracias al uso de software libre y contenedores Docker, la gran mayoría de la inversión reside en el capital humano y el desarrollo intelectual, minimizando los costes de mantenimiento estructural.



8.- CONCLUSIONES

Tras varios meses de desarrollo continuo a lo largo de los diferentes Sprints, el proyecto SinergyFit ha llegado a su versión estable y desplegable. A modo de cierre, se exponen las siguientes reflexiones y valoraciones sobre el resultado obtenido.

8.1. GRADO DE CUMPLIMIENTO DE LOS OBJETIVOS

Los objetivos principales establecidos al inicio del proyecto se han cumplido en su totalidad. Se ha logrado desarrollar una plataforma que permite a los usuarios registrar sus entrenamientos de forma intuitiva, calcular sus métricas de progreso automáticamente y visualizar su rendimiento a través de un panel de control interactivo. La transición de un sistema en PHP puro a una arquitectura robusta basada en Laravel ha dotado al proyecto de una profesionalidad técnica superior a la exigida inicialmente.

8.2. DIFICULTADES ENCONTRADAS

Durante el desarrollo, los mayores retos técnicos no estuvieron relacionados con el diseño visual, sino con la lógica de servidor y la infraestructura:

- La curva de aprendizaje del ORM Eloquent: Migrar la mentalidad de consultas SQL puras (PDO) a colecciones y modelos orientados a objetos requirió un esfuerzo de comprensión importante durante el Sprint 5.
- La gestión de dependencias y el despliegue: Configurar el entorno Docker (docker-compose) para asegurar la comunicación fluida entre el contenedor web y la base de datos presentó desafíos de configuración de red y volúmenes, que finalmente fueron solventados.

8.3. AMPLIACIONES FUTURAS

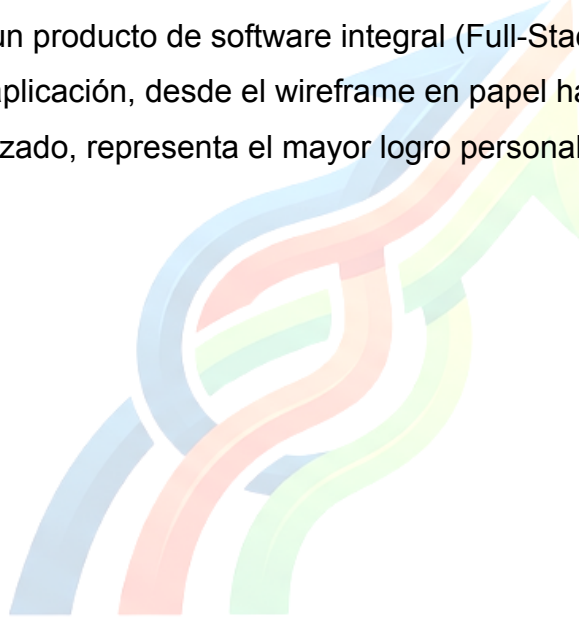
SinergyFit se ha diseñado como una base sólida y escalable. Aunque la versión actual es completamente funcional, existen varias vías de expansión a futuro:

- Desarrollo de una API REST pública para conectar la base de datos con una futura aplicación móvil nativa (Android/iOS).

- Módulo social: Permitir a los usuarios compartir sus rutinas y logros con amigos.
- Integración de Inteligencia Artificial: Desarrollar un algoritmo que sugiera rutinas de entrenamiento basadas en el progreso histórico del usuario.

8.4. VALORACIÓN PERSONAL

A nivel académico y profesional, el desarrollo de este proyecto intermodular ha sido inmensamente enriquecedor. Ha servido como puente entre el conocimiento teórico fragmentado (aprender bases de datos por un lado, y maquetación por otro) y la construcción real de un producto de software integral (Full-Stack). Comprender el ciclo de vida de una aplicación, desde el wireframe en papel hasta su ejecución en un contenedor virtualizado, representa el mayor logro personal de este curso.



9.- BIBLIOGRAFÍA Y WEBGRAFÍA

El desarrollo de SinergyFit no habría sido posible sin el apoyo de la extensa documentación técnica de las tecnologías de código abierto utilizadas. Las principales fuentes de consulta han sido:

- Documentación Oficial de Laravel (v10.x): Referencia principal para el enrutamiento, Eloquent ORM, Blade y autenticación. <https://laravel.com/docs>
- Documentación Oficial de PHP: Consultas sobre sintaxis y funciones nativas. <https://www.php.net/manual/es/>
- Bootstrap 5 Docs: Guías de componentes visuales y utilidades del sistema de grillas responsivo. <https://getbootstrap.com/docs/5.0/getting-started/introduction/>
- Chart.js Documentation: Implementación y configuración de gráficos interactivos. <https://www.chartjs.org/docs/latest/>
- Docker Docs: Referencias para la creación de imágenes y archivos de orquestación (Docker Compose). <https://docs.docker.com/>

10.- GLOSARIO

Para facilitar la comprensión técnica del documento, se definen brevemente los siguientes conceptos clave utilizados a lo largo de la memoria:

- API (Application Programming Interface): Conjunto de reglas que permiten que diferentes aplicaciones se comuniquen entre sí.
- CRUD (Create, Read, Update, Delete): Las cuatro operaciones básicas que se pueden realizar sobre una base de datos (Crear, Leer, Actualizar y Borrar).
- Contenedor / Docker: Tecnología de virtualización ligera que empaqueta una aplicación y todas sus dependencias en una sola unidad estandarizada para facilitar su despliegue.
- Eloquent (ORM): El Mapeador Objeto-Relacional incluido en Laravel. Permite interactuar con la base de datos utilizando sintaxis orientada a objetos en lugar de escribir consultas SQL tradicionales.
- MVC (Modelo-Vista-Controlador): Patrón de arquitectura de software que separa los datos de una aplicación, la interfaz de usuario y la lógica de control en tres componentes distintos.
- UI/UX (User Interface / User Experience): Diseño de la Interfaz de Usuario y Experiencia de Usuario. Se refiere a cómo se ve la aplicación visualmente y lo fácil o intuitivo que resulta usarla.

11.- ANEXOS

Manual de Usuario - SinergyFit

Bienvenido a SinergyFit. Esta guía te ayudará a navegar por la plataforma y sacar el máximo provecho al registro de tus entrenamientos físicos.

1. Registro e Inicio de Sesión

- Registro: Accede a la pantalla inicial y haz clic en "Registrarse". Introduce tu nombre, tu correo electrónico y una contraseña segura.
- Login: Si ya dispones de una cuenta, simplemente ingresa tus credenciales en la pantalla de inicio de sesión.

2. El Dashboard (Panel Principal)

Nada más entrar a la plataforma, visualizarás tu Dashboard.

- Tarjetas de Resumen: En la parte superior verás métricas rápidas (ej. total de minutos entrenados esta semana).
- Gráficas de Rendimiento: Un gráfico interactivo te mostrará tu progresión temporal de los últimos 7 días.

3. Registrar un Entrenamiento

Para anotar lo que has hecho hoy, sigue estos pasos:

1. Ve a "Nuevo Entrenamiento" en el menú lateral izquierdo.
2. Selecciona el Tipo de Ejercicio (Fuerza, Caminata o Carrera).
3. Si seleccionas Caminata o Carrera: Solo tendrás que introducir la duración total en minutos.
4. Si seleccionas Fuerza: Se desplegará un formulario dinámico. Haz clic en "Añadir Serie" tantas veces como necesites. Por cada serie, indica el nombre del ejercicio (ej. Press Banca), el peso (Kg) y las repeticiones.
5. Haz clic en "Guardar Entrenamiento" para subirlo al servidor.

4. Visualización en Calendario

En el apartado "Calendario" del menú lateral, dispones de una vista mensual completa.

- Cada entrenamiento que hayas guardado aparecerá como una etiqueta de color (ej. Rojo para Fuerza, Azul para Cardio).
- Puedes hacer clic en cualquier etiqueta para ver los detalles exactos de lo que hiciste ese día.

5. Gestión de Metas y Objetivos

1. Ve a la sección "Mis Metas".
2. Haz clic en "Crear Meta" (por ejemplo: Levantar 100kg en Sentadilla o Correr 10km sin parar).
3. Establece una fecha límite realista para tu meta.
4. Cuando la consigas en la vida real, vuelve aquí y marca la casilla de "Completado" para llevar tu seguimiento personal.

6. Perfil, Biometría y Gamificación (Logros)

- En la esquina superior derecha, haz clic en tu nombre y ve a "Mi Perfil".
- Biometría: Aquí puedes actualizar tu Peso corporal, tu Porcentaje de Grasa y subir una foto de avatar personalizada.
- Vitrina de Logros: Justo debajo de tus datos, verás tu vitrina de medallas. A medida que entrenes (ej. Registrar tu primer entrenamiento, llegar a 1000 minutos de cardio), el sistema detectará tus hitos y te otorgará medallas automáticamente. Las medallas que aún no hayas conseguido aparecerán bloqueadas (en color gris oscuro).

Manual de Instalación - SinergyFit

Requisitos Previos

Para instalar y ejecutar SinergyFit en un entorno local, asegúrate de cumplir con los siguientes requisitos del sistema:

- XAMPP / Laragon / MAMP (o cualquier entorno con PHP y MySQL).
- PHP $\geq$ 8.1
- MySQL $\geq$ 8.0 o MariaDB.
- Composer (Gestor de dependencias de PHP).
- Node.js y NPM (Para compilar assets de frontend si fuera necesario).
- Git (Para clonar el repositorio).

Pasos de Instalación

1. Clonar el repositorio

Abre tu terminal o consola y dirígete al directorio de tu servidor local (ej. C:\xampp\htdocs en Windows).

```
git clone https://github.com/tu-usuario/fitnesstracker-laravel.git
cd fitnesstracker-laravel
```

2. Instalar dependencias de PHP (Composer)

Ejecuta el siguiente comando para descargar las librerías del core de Laravel:
composer install

3. Instalar dependencias de Frontend (NPM)

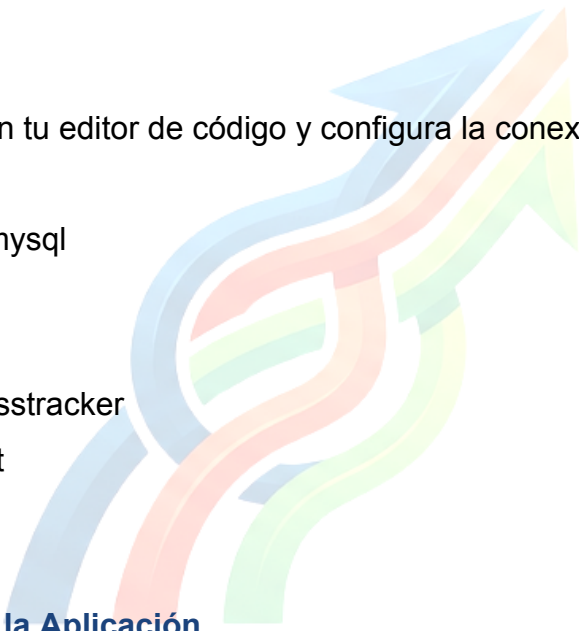
```
npm install  
npm run build
```

4. Configuración del Entorno (.env)

Copia el archivo de ejemplo para crear tu entorno local:
cp .env.example .env

Abre el archivo .env en tu editor de código y configura la conexión a la base de datos:

```
DB_CONNECTION=mysql  
DB_HOST=127.0.0.1  
DB_PORT=3306  
DB_DATABASE=fitnesstracker  
DB_USERNAME=root  
DB_PASSWORD=
```



5. Generar la Key de la Aplicación

Laravel requiere una clave de encriptación segura. Genérala con:
php artisan key:generate

6. Migraciones y Seeders (Base de Datos)

Asegúrate de haber creado una base de datos vacía llamada fitnesstracker en tu gestor (ej. phpMyAdmin). Luego ejecuta las migraciones para crear las tablas y volcar datos iniciales:

```
php artisan migrate --seed
```

7. Enlazar el Almacenamiento Público

Para que la subida de fotos de perfil funcione, crea el enlace simbólico:
php artisan storage:link

8. Iniciar el Servidor Local

Finalmente, levanta el servidor de desarrollo:

```
php artisan serve
```

Accede en tu navegador a: <http://localhost:8000>

- Anexo II: Diagramas técnicos a tamaño completo (Si el diagrama E/R no se veía bien en el texto, lo pones aquí en grande).
- Anexo III: Fragmentos de código relevantes (Ej. El contenido completo de tu archivo docker-compose.yml o del Controlador de Entrenamientos, para que vean cómo programas).

